Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Segundo Ciclo, 2026

Catedrático: Erick Marroquin

Laboratorio 4

Polimorfismo y

Nombre: Emmanuel Montufar y Daniel Chou-Jo

Carné: 26805 y 26968

Sección # 30

Fecha de entrega:

06/11/2026

Análisis

Requisitos funcionales

1. El programa debe permitir ingresar nuevos equipos con código único y no vacío, nombre y empresa.

2. El programa debe permitir agregar nuevos deportes mediante subclases, sin modificar la tabla de posiciones ni el calendario.

3. El programa debe rechazar la programación o reprogramación de eventos cuando exista un conflicto de sede, fecha y hora, o cuando un equipo tenga otro encuentro en esa misma fecha y hora.

4. El programa debe permitir crear exhibiciones con título, sin equipos, marcador ni resultado.

5. El programa debe incluir los deportes de baloncesto y fútbol sala.

6. El programa debe mostrar un calendario con los eventos que tienen programación, incluidos los encuentros finalizados.

7. El programa debe calcular los puntos de tabla de cada encuentro según el deporte y mostrar la tabla por puntos descendentes y, en caso de empate, por nombre alfabético.

8. El programa debe implementar dos interfaces distintas: ICalendario para la programación y el calendario, e IResultado para registrar resultados y repartir puntos.

9. El programa debe implementar una clase abstracta Encuentros con los datos comunes y una subclase concreta por deporte.

10. Exhibicion debe ser una clase concreta que implemente únicamente ICalendario.

11. El programa debe implementar un solo criterio de orden cronológico por fecha, hora y nombre de sede.

Reglas de cada deporte y cómo se solucionan.

Fut Sala

El marcador son los goles de cada equipo. Se permiten empates. La victoria otorga 3 puntos de tabla, el empate 1 punto a cada equipo y la derrota 0 puntos.

Baloncesto

El marcador son los puntos anotados por cada equipo. No se permiten empates: un marcador igual se rechaza. La victoria otorga 2 puntos de tabla y la derrota 1 punto.

Solución

Los equipos, el marcador, el estado y la programación se conservan en la clase abstracta Encuentros. FutSal y Baloncesto sobrescriben los métodos para validar el marcador y calcular los puntos de tabla según las reglas de cada deporte.

Tabla de estados

Objeto	*Estado Actual*	*Operación*	*Condiciones para realizarla*	*Estado Resultante*
Encuentro	No existe	Crear encuentro	Seleccionar un deporte y dos equipos registrados y distintos.	Sin programar
Encuentro	Sin programar	Programar	Fecha válida, hora de 0 a 23 y sede registrada disponible. Ningún equipo tiene otro encuentro en esa fecha y hora.	Programado
Encuentro	Programado	Reprogramar	Validar nueva fecha, hora, sede y disponibilidad de equipos, excluyendo al propio encuentro del conflicto.	Programado
Encuentro	Programado	Registrar resultado	Marcadores enteros no negativos. Baloncesto rechaza empates; fútbol sala los permite. Sumar puntos una sola vez.	Finalizado
Encuentro	Sin programar	Intentar registrar resultado	Operación rechazada: primero debe programarse. No se modifica ningún dato.	Sin programar
Encuentro	Finalizado	Intentar registrar otro resultado	Operación rechazada: ya tiene resultado. Conservar marcador y puntos.	Finalizado
Encuentro	Finalizado	Intentar reprogramar	Operación rechazada: no puede reprogramarse un encuentro con resultado.	Finalizado
Encuentro	Sin programar	Eliminar	El encuentro debe existir y no tener resultado.	Eliminado de la copa
Encuentro	Programado	Eliminar	Debe existir y no tener resultado. Retirarlo del calendario y liberar su sede y horario para los equipos.	Eliminado de la copa
Encuentro	Finalizado	Intentar eliminar	Operación rechazada: conserva el resultado y los puntos otorgados.	Finalizado
Exhibición	No existe	Crear exhibición	Ingresar un título no vacío.	Sin programar
Exhibición	Sin programar	Programar	Fecha válida, hora de 0 a 23 y sede registrada disponible en esa fecha y hora.	Programada
Exhibición	Programada	Reprogramar	Validar la nueva programación y excluir a la propia exhibición de los conflictos.	Programada
Exhibición	Sin programar	Eliminar	La exhibición debe existir.	Eliminada de la copa
Exhibición	Programada	Eliminar	Debe existir. Retirarla del calendario y liberar su sede y horario.	Eliminada de la copa

Clases

Clases modelo

Encuentros <<Abstract>>

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	id	int	private final	Identificador recibido desde Copa.
2	equipo1	Equipo	private final	Referencias a dos equipos registrados y distintos.
3	equipo2	Equipo	private final	Referencias a dos equipos registrados y distintos.
4	marcadorE1	int	private	Valores del resultado, compartidos por los deportes. Inician en 0; el estado indica si ya existe resultado.
5	marcadorE2	int	private	Valores del resultado, compartidos por los deportes. Inician en 0; el estado indica si ya existe resultado.
6	estado	EstadoEncuentro	private	Inicia en SIN_PROGRAMAR.
7	fecha	LocalDate	private	Inicia en null.
8	hora	int	private	Inicia en -1; al programar admite de 0 a 23.
9	sede	String	private	Inicia en null.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	Encuentros	protected	int id, Equipo equipo1, Equipo equipo2	No aplica: constructor	Constructor. Validar id y equipos distintos y no nulos. Inicializar los demás datos; no recibe marcador ni programación.
2	getEquipo1	public	Ninguno	Equipo	Consultar participantes.
3	getEquipo2	public	Ninguno	Equipo	Consultar participantes.
4	getMarcadorE1	public	Ninguno	int	Consultar marcadores solo si el estado es FINALIZADO; en otro caso lanzar IllegalStateException.
5	getMarcadorE2	public	Ninguno	int	Consultar marcadores solo si el estado es FINALIZADO; en otro caso lanzar IllegalStateException.
6	getMarcador	public	Ninguno	String	Producir una representación textual del resultado; mostrar Sin resultado cuando no está finalizado. No guardar otro String con el mismo dato.
7	getEstado	public	Ninguno	EstadoEncuentro	Consultar el estado actual.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
8	getId	public	Ninguno	int	Identificar el evento de manera única.
9	getFecha	public	Ninguno	LocalDate	Consultar la fecha; devuelve null si no está programado.
10	getHora	public	Ninguno	int	Consultar la hora; devuelve -1 si no está programado.
11	getSede	public	Ninguno	String	Consultar la sede; devuelve null si no está programado.
12	estaProgramado	public	Ninguno	boolean	Indicar si tiene programación. Devuelve true también para un encuentro finalizado.
13	getDescripcion	public	Ninguno	String	Proporcionar los equipos del encuentro o el título de la exhibición sin preguntar por su tipo.
14	programar	public	LocalDate fecha, int hora, String sede	void	Asignar inicialmente una programación válida y cambiar el estado correspondiente.
15	reprogramar	public	LocalDate fecha, int hora, String sede	void	Cambiar la programación cuando el estado lo permite.
16	registrarResultado	public final	int marcadorE1, int marcadorE2	void	Exigir PROGRAMADO y marcador válido; calcular ambos puntajes antes de modificar datos. Guardar resultado, acumular puntos y pasar a FINALIZADO.
17	validarMarcador	protected abstract	int marcadorE1, int marcadorE2	boolean	Delegar la validación a cada deporte.
18	calcularPuntosEquipo1	protected abstract	int marcadorE1, int marcadorE2	int	Determinar los puntos de tabla del primer equipo para un marcador válido.
19	calcularPuntosEquipo2	protected abstract	int marcadorE1, int marcadorE2	int	Determinar los puntos de tabla del segundo equipo para un marcador válido.

Clase Hija 1 (FutSal)

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	PUNTOS_GANADOR	int	private static final	Puntos por victoria: 3.
2	PUNTOS_PERDEDOR	int	private static final	Puntos por derrota: 0.
3	PUNTOS_EMPATE	int	private static final	Puntos por empate: 1 a cada equipo.

Métodos FutSal

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	FutSal	public	int id, Equipo equipo1, Equipo equipo2	No aplica: constructor	Constructor. Llamar a super(id, equipo1, equipo2) e iniciar sin programar y sin resultado.
2	validarMarcador	protected	int marcadorE1, int marcadorE2	boolean	Aceptar dos marcadores enteros no negativos; permite empates.
3	calcularPuntosEquipo1	protected	int marcadorE1, int marcadorE2	int	Sobrescribir el método abstracto y devolver para el equipo 1: 3 si gana, 1 si empata y 0 si pierde.
4	calcularPuntosEquipo2	protected	int marcadorE1, int marcadorE2	int	Sobrescribir el método abstracto y devolver para el equipo 2: 3 si gana, 1 si empata y 0 si pierde.

Clase Hija 2 (Baloncesto)

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	PUNTOS_GANADOR	int	private static final	Puntos por victoria: 2.
2	PUNTOS_PERDEDOR	int	private static final	Puntos por derrota: 1.

Métodos Baloncesto

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	Baloncesto	public	int id, Equipo equipo1, Equipo equipo2	No aplica: constructor	Constructor. Llamar a super(id, equipo1, equipo2) e iniciar sin programar y sin resultado.
2	validarMarcador	protected	int marcadorE1, int marcadorE2	boolean	Aceptar dos marcadores enteros no negativos y diferentes; rechazar empates.
3	calcularPuntosEquipo1	protected	int marcadorE1, int marcadorE2	int	Sobrescribir el método abstracto y devolver para el equipo 1: 2 si gana y 1 si pierde, después de validar que no hay empate.
4	calcularPuntosEquipo2	protected	int marcadorE1, int marcadorE2	int	Sobrescribir el método abstracto y devolver para el equipo 2: 2 si gana y 1 si pierde, después de validar que no hay empate.

Clase Exhibicion

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	id	int	private final	Identificador asignado por Copa.

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
2	titulo	String	private	Nombre de la clínica o demostración, no vacío.
3	estado	EstadoExhibicion	private	Inicia en SIN_PROGRAMAR.
4	fecha	LocalDate	private	Inicia en null.
5	hora	int	private	Inicia en -1.
6	sede	String	private	Inicia en null.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	Exhibicion	public	int id, String titulo	No aplica: constructor	Validar datos e iniciar sin programar. No recibir estado ni programación.
2	getTitulo	public	Ninguno	String	Consultar el título.
3	getEstado	public	Ninguno	EstadoExhibicion	Consultar el estado.
4	setTitulo	public	String titulo	void	Opcional: cambiar el título, rechazando valores vacíos. No altera programación ni estado.
5	getId	public	Ninguno	int	Identificar el evento de manera única.
6	getFecha	public	Ninguno	LocalDate	Consultar la fecha; devuelve null si no está programado.
7	getHora	public	Ninguno	int	Consultar la hora; devuelve -1 si no está programado.
8	getSede	public	Ninguno	String	Consultar la sede; devuelve null si no está programado.
9	estaProgramado	public	Ninguno	boolean	Indicar si tiene programación. Devuelve true también para un encuentro finalizado.
10	getDescripcion	public	Ninguno	String	Proporcionar los equipos del encuentro o el título de la exhibición sin preguntar por su tipo.
11	programar	public	LocalDate fecha, int hora, String sede	void	Asignar inicialmente una programación válida y cambiar el estado correspondiente.
12	reprogramar	public	LocalDate fecha, int hora, String sede	void	Cambiar la programación cuando el estado lo permite.

Clase Equipo

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	codigo	String	private final	Código único e inmutable.

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
2	nombre	String	private	Nombre no vacío.
3	empresa	String	private	Empresa representada, no vacía.
4	puntos	int	private	Puntos de tabla; inician en cero.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	Equipo	public	String codigo, String nombre, String empresa	No aplica: constructor	Validar los datos e iniciar puntos en cero. Copa comprueba que el código no esté repetido.
2	getCodigo	public	Ninguno	String	Consultar la información del equipo.
3	getNombre	public	Ninguno	String	Consultar la información del equipo.
4	getEmpresa	public	Ninguno	String	Consultar la información del equipo.
5	getPuntos	public	Ninguno	int	Devolver los puntos como int, no String.
6	sumarPuntos	De paquete	int cantidad	void	Acumular una cantidad no negativa desde el registro de resultados. Mantener Equipo y Encuentros en el mismo paquete del modelo; no exponer asignación libre desde la vista.
7	compareTo	public	Equipo otro	int	Comparar puntos descendentes y luego nombres con String.compareTo. No incluir goles ni marcadores como criterio de desempate.
8	setNombre	public	String nombre	void	Opcionales: actualizar texto no vacío. El código y los puntos no tienen setters públicos.
9	setEmpresa	public	String empresa	void	Opcionales: actualizar texto no vacío. El código y los puntos no tienen setters públicos.

Clase Copa

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	nombre	String	private final	Nombre de la copa.
2	equipos	List<Equipo>	private final	Equipos registrados.
3	eventos	List<ICalendario>	private final	Todos los encuentros y exhibiciones, incluso sin programar.
4	sedes	Set<String>	private final	Catálogo de sedes; aplicar la normalización acordada para detectar duplicados.
5	siguienteIdEvento	int	private	Contador que inicia en 1. Genera identificadores y no reutiliza los eliminados.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	Copa	public	String nombre	No aplica: constructor	Validar el nombre e inicializar colecciones y contador.
2	agregarEquipo	public	String codigo, String nombre, String empresa	Equipo	Validar unicidad, crear el equipo y registrarlo.
3	buscarEquipo	public	String codigo	Equipo	Buscar un equipo o consultar todos. Informar identificadores inexistentes.
4	listarEquipos	public	Ninguno	List<Equipo>	Buscar un equipo o consultar todos. Informar identificadores inexistentes.
5	eliminarEquipo	public	String codigo	void	Exigir existencia y ausencia de relaciones con encuentros registrados.
6	agregarSede	public	String nombreSede	void	Gestionar el catálogo. Rechazar duplicados, vacíos o eliminación de sedes asignadas.
7	eliminarSede	public	String nombreSede	void	Gestionar el catálogo. Rechazar duplicados, vacíos o eliminación de sedes asignadas.
8	listarSedes	public	Ninguno	List<String>	Obtener las sedes registradas.
9	crearFutSal	public	String codigoEquipo1, String codigoEquipo2	FutSal	Buscar participantes, asignar id y registrar un encuentro válido sin programar.
10	crearBaloncesto	public	String codigoEquipo1, String codigoEquipo2	Baloncesto	Buscar participantes, asignar id y registrar un encuentro válido sin programar.
11	crearExhibicion	public	String titulo	Exhibicion	Asignar id y registrar una exhibición válida sin programar.
12	buscarEncuentro	public	int id	Encuentros	Resolver identificadores por la operación correspondiente y rechazar inexistentes.
13	buscarExhibicion	public	int id	Exhibicion	Resolver identificadores por la operación correspondiente y rechazar inexistentes.
14	listarEncuentros	public	Ninguno	List<Encuentros>	Permitir consultar cada categoría sin perder sus eventos sin programar.
15	listarExhibiciones	public	Ninguno	List<Exhibicion>	Permitir consultar cada categoría sin perder sus eventos sin programar.
16	programarEvento	public	int id, LocalDate fecha, int hora, String sede	void	Comprobar datos, estado y conflictos globales. Solo después solicitar la programación al evento.
17	reprogramarEvento	public	int id, LocalDate fecha, int hora, String sede	void	Validar la nueva programación excluyendo el propio evento y reemplazar datos solo si todo es válido.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
18	eliminarEncuentro	public	int id	void	Aplicar las restricciones de eliminación y retirar el evento de la colección y de los índices que correspondan.
19	eliminarExhibicion	public	int id	void	Aplicar las restricciones de eliminación y retirar el evento de la colección y de los índices que correspondan.
20	obtenerTablaPosiciones	public	Ninguno	List<Equipo>	Ordenar una copia de los equipos utilizando Comparable<Equipo>.
21	obtenerCalendario	public	Ninguno	List<ICalendario>	Filtrar los eventos con programación y ordenar una copia utilizando el criterio de ICalendario.
22	iterator	public	Ninguno	Iterator<ICalendario>	Recorrer el calendario ordenado sin permitir eliminar eventos mediante el iterador.
23	generarIdEvento	private	Ninguno	int	Entregar el siguiente identificador y avanzar el contador.
24	validarDisponibilidad	private	ICalendario evento, LocalDate fecha, int hora, String sede	void	Centralizar conflictos de sede y, para encuentros, de equipos. Excluir el mismo id al reprogramar.

Controladores

Controlador Copa

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	copa	Copa	private final	Referencia a la copa que contiene los equipos, las sedes y los eventos. Se recibe en el constructor y se comparte con los demás controladores.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	ControladorCopa	public	Copa copa	Constructor	Guardar la instancia compartida de Copa. Rechazar una referencia nula. No tiene tipo de retorno.
2	agregarEquipo	public	String codigo, String nombre, String empresa	Equipo	Solicitar la creación y el registro de un equipo. Rechazar datos vacíos y códigos repetidos. El equipo inicia con cero puntos de tabla. Devolver el equipo registrado.
3	eliminarEquipo	public	String codigo	void	Solicitar la eliminación de un equipo existente. Rechazarla si participa en algún encuentro registrado, cualquiera que sea su estado, para conservar las referencias y los resultados.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
4	agregarSede	public	String nombreSede	void	Solicitar el registro de una sede. Rechazar nombres vacíos o repetidos según el criterio de identificación adoptado.
5	eliminarSede	public	String nombreSede	void	Solicitar la eliminación de una sede existente. Rechazarla si está asignada a algún evento, incluidos los encuentros finalizados.
6	listarSedes	public	Ninguno	List<String>	Obtener las sedes registradas para mostrarlas o seleccionar una al programar un evento.
7	obtenerTablaPosiciones	public	Ninguno	List<Equipo>	Obtener todos los equipos ordenados por puntos descendentes y luego por nombre alfabético. Incluir equipos con cero puntos.
8	obtenerCalendario	public	Ninguno	List<ICalendario>	Obtener los encuentros y exhibiciones con programación, incluidos los encuentros finalizados. Ordenar por fecha, hora y nombre de sede; excluir eventos sin programar.

ControladorEncuentros

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	copa	Copa	private final	Referencia a la instancia compartida de Copa para acceder a los equipos, sedes y eventos registrados. Los marcadores, estados y equipos participantes se conservan en cada encuentro.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	ControladorEncuentros	public	Copa copa	Constructor	Guardar la instancia compartida de Copa. Rechazar una referencia nula. No tiene tipo de retorno.
2	crearFutSal	public	String codigoEquipo1, String codigoEquipo2	FutSal	Buscar dos equipos registrados y distintos. Crear y registrar el encuentro con identificador único, sin programar y sin resultado. Rechazar equipos inexistentes o iguales.
3	crearBaloncesto	public	String codigoEquipo1, String codigoEquipo2	Baloncesto	Buscar dos equipos registrados y distintos. Crear y registrar el encuentro con identificador único, sin programar y sin resultado. Rechazar equipos inexistentes o iguales.
4	buscarEncuentro	public	int idEncuentro	Encuentros	Obtener el encuentro por su identificador. Informar si el identificador no corresponde a un encuentro registrado.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
5	listarEncuentros	public	Ninguno	List<Encuentros>	Obtener todos los encuentros sin programar, programados y finalizados para consultarlos o seleccionar uno.
6	programarEncuentro	public	int idEncuentro, LocalDate fecha, int hora, String sede	void	Solicitar la programación de un encuentro sin programar. Validar todos los datos y conflictos antes de guardar fecha, hora y sede y pasar a programado.
7	reprogramarEncuentro	public	int idEncuentro, LocalDate nuevaFecha, int nuevaHora, String nuevaSede	void	Solicitar el cambio de programación de un encuentro programado y sin resultado. Validar los nuevos datos excluyendo al propio encuentro de los conflictos. Conservar la programación anterior si falla.
8	registrarResultado	public	int idEncuentro, int marcadorE1, int marcadorE2	void	Buscar el encuentro y llamar a registrarResultado. El modelo exige que esté programado, valida el marcador por deporte, guarda el resultado, suma puntos una sola vez y pasa a finalizado.
9	eliminarEncuentro	public	int idEncuentro	void	Solicitar la eliminación de un encuentro sin programar o programado. Retirarlo también del calendario si corresponde. Rechazar la eliminación de un encuentro finalizado.

ControladorExhibiciones

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	copa	Copa	private final	Referencia a la instancia compartida de Copa para acceder a exhibiciones, sedes y demás eventos y solicitar las comprobaciones de disponibilidad.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	ControladorExhibiciones	public	Copa copa	Constructor	Guardar la instancia compartida de Copa. Rechazar una referencia nula. No tiene tipo de retorno.
2	crearExhibicion	public	String titulo	Exhibicion	Crear y registrar una exhibición con identificador único y estado sin programar. Rechazar un título vacío o compuesto únicamente por espacios.
3	buscarExhibicion	public	int idExhibicion	Exhibicion	Obtener una exhibición por su identificador. Informar si no corresponde a una exhibición registrada.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
4	listarExhibiciones	public	Ninguno	List<Exhibicion>	Obtener todas las exhibiciones, tanto sin programar como programadas, para consultarlas o seleccionar una.
5	programarExhibicion	public	int idExhibicion, LocalDate fecha, int hora, String sede	void	Solicitar la programación de una exhibición sin programar. Validar fecha, hora de 0 a 23, sede registrada y ausencia de otro evento en esa sede, fecha y hora. Pasar a programada.
6	reprogramarExhibicion	public	int idExhibicion, LocalDate nuevaFecha, int nuevaHora, String nuevaSede	void	Solicitar el cambio de programación de una exhibición programada. Excluirla de sus propios conflictos y conservar los datos anteriores si la validación falla. Mantener el estado programada.
7	eliminarExhibicion	public	int idExhibicion	void	Solicitar la eliminación de una exhibición existente en cualquiera de sus dos estados. Si estaba programada, retirarla del calendario y dejar disponible su sede en esa fecha y hora.

ControladorEquipos

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	copa	Copa	private final	Referencia a la instancia compartida de Copa para consultar los equipos y sus puntos actuales. No mantiene una colección adicional de equipos.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	ControladorEquipos	public	Copa copa	Constructor	Guardar la instancia compartida de Copa. Rechazar una referencia nula. No tiene tipo de retorno.
2	buscarEquipo	public	String codigo	Equipo	Obtener un equipo por su código único. Permitir consultar nombre, empresa, código y puntos mediante sus getters. Informar si no existe.
3	listarEquipos	public	Ninguno	List<Equipo>	Obtener los equipos registrados para mostrarlos o seleccionarlos al crear un encuentro. No exige el orden de la tabla de posiciones.
4	consultarPuntos	public	String codigo	int	Consultar los puntos de tabla acumulados por un equipo existente. Los puntos proceden de resultados válidos de encuentros finalizados.

Clase Vista

Atributos

No.	*Atributo*	*Tipo*	*Visibilidad*	*Propósito*
1	entrada	Scanner	private final	Leer las entradas desde la consola.
2	controladorCopa	ControladorCopa	private final	Solicitar registro y eliminación de equipos y sedes y consultar tabla y calendario.
3	controladorEncuentros	ControladorEncuentros	private final	Solicitar las operaciones sobre encuentros.
4	controladorExhibiciones	ControladorExhibiciones	private final	Solicitar las operaciones sobre exhibiciones.
5	controladorEquipos	ControladorEquipos	private final	Consultar equipos y sus puntos.

Métodos

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
1	Vista	public	Scanner entrada, ControladorCopa controladorCopa, ControladorEncuentros controladorEncuentros, ControladorExhibiciones controladorExhibiciones, ControladorEquipos controladorEquipos	No aplica: constructor	Recibir y guardar el lector y los cuatro controladores; rechazar referencias nulas.
2	iniciar	public	Ninguno	void	Mantener los menús hasta elegir salir. Capturar errores por operación y mostrar su motivo sin terminar inesperadamente.
3	mostrarMenuPrincipal	private	Ninguno	void	Mostrar las opciones principales y la opción de salir.
4	menuEquipos	private	Ninguno	void	Gestionar registro, eliminación, listado y consulta de equipos.
5	menuSedes	private	Ninguno	void	Gestionar registro, eliminación y listado de sedes.
6	menuEncuentros	private	Ninguno	void	Gestionar creación, consultas, programación, resultados y eliminación de encuentros.
7	menuExhibiciones	private	Ninguno	void	Gestionar creación, consultas, programación y eliminación de exhibiciones.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
8	registrarEquipo	private	Ninguno	void	Solicitar código, nombre y empresa y llamar a controladorCopa.agregarEquipo.
9	eliminarEquipo	private	Ninguno	void	Solicitar código y llamar a controladorCopa.eliminarEquipo.
10	mostrarEquipos	private	Ninguno	void	Consultar controladorEquipos.listarEquipos y mostrar sus datos; indicar si no hay equipos.
11	consultarEquipo	private	Ninguno	void	Solicitar código y mostrar los datos y puntos del equipo obtenido por buscarEquipo.
12	agregarSede	private	Ninguno	void	Solicitar nombre y llamar a controladorCopa.agregarSede.
13	eliminarSede	private	Ninguno	void	Solicitar nombre y llamar a controladorCopa.eliminarSede.
14	mostrarSedes	private	Ninguno	void	Consultar y mostrar controladorCopa.listarSedes; indicar si no hay registros.
15	crearEncuentro	private	Ninguno	void	Solicitar deporte y códigos de dos equipos; llamar a crearFutSal o crearBaloncesto y mostrar el id asignado.
16	mostrarEncuentros	private	Ninguno	void	Mostrar id, participantes y estado de todos los encuentros; incluir programación y resultado cuando existan.
17	consultarEncuentro	private	Ninguno	void	Solicitar id y mostrar los detalles del encuentro obtenido por buscarEncuentro.
18	programarEncuentro	private	Ninguno	void	Solicitar id, fecha, hora y sede y llamar al controlador.
19	reprogramarEncuentro	private	Ninguno	void	Solicitar id y nueva fecha, hora y sede y llamar al controlador.
20	registrarResultado	private	Ninguno	void	Solicitar id y dos marcadores enteros no negativos y llamar al controlador.
21	eliminarEncuentro	private	Ninguno	void	Solicitar id y llamar a eliminarEncuentro del controlador.
22	crearExhibicion	private	Ninguno	void	Solicitar título y llamar a crearExhibicion; mostrar el id asignado.
23	mostrarExhibiciones	private	Ninguno	void	Listar id, título, estado y programación cuando exista.
24	consultarExhibicion	private	Ninguno	void	Solicitar id y mostrar los detalles obtenidos por buscarExhibicion.
25	programarExhibicion	private	Ninguno	void	Solicitar id, fecha, hora y sede y llamar al controlador.

No.	*Método*	*Visibilidad*	*Parámetros*	*Retorno*	*Propósito*
26	reprogramarExhibicion	private	Ninguno	void	Solicitar id y nueva programación y llamar al controlador.
27	eliminarExhibicion	private	Ninguno	void	Solicitar id y llamar a eliminarExhibicion del controlador.
28	mostrarTablaPosiciones	private	Ninguno	void	Obtener la tabla y mostrar posición, código, nombre, empresa y puntos, respetando el orden recibido.
29	mostrarCalendario	private	Ninguno	void	Obtener el calendario y mostrar fecha, hora, sede y getDescripcion de cada evento; respetar el orden recibido sin preguntar por su tipo.
30	leerTextoObligatorio	private	String mensaje	String	Leer y quitar espacios extremos; repetir si el texto queda vacío.
31	leerEnteroEnRango	private	String mensaje, int minimo, int maximo	int	Leer un entero en el rango y repetir ante formato incorrecto, desbordamiento o valor fuera del rango.
32	leerFecha	private	String mensaje	LocalDate	Leer una fecha aaaa-MM-dd y repetir ante formato incorrecto o fecha inexistente.

Justifique por qué el calendario y la tabla de posiciones se apoyan en dos interfaces distintas en lugar de una sola, y qué pasaría si las exhibiciones tuvieran que cumplir la interfaz de la tabla.

Se utilizan dos interfaces porque la programación de eventos y el registro de resultados tienen responsabilidades diferentes. ICalendario permite programar y mostrar tanto encuentros como exhibiciones. IResultado permite registrar marcadores y repartir puntos de tabla, por lo que únicamente la implementan los encuentros.

Si las exhibiciones tuvieran que cumplir la interfaz de resultados, estarían obligadas a implementar operaciones que no les corresponden, porque no tienen equipos, marcadores ni puntos. Esto produciría métodos vacíos o que lanzarían errores. Separar las interfaces permite que cada clase implemente únicamente el comportamiento que necesita.

Explique qué tendría que crear o modificar para agregar un tercer deporte, y si la tabla de posiciones y el calendario tendrían que cambiar.

Para agregar un tercer deporte se debe crear una nueva subclase de Encuentros, por ejemplo, Voleibol. Esta heredaría los equipos, el marcador, el estado y la programación, y sobrescribiría los métodos validarMarcador, calcularPuntosEquipo1 y calcularPuntosEquipo2 con sus propias reglas. Su constructor llamaría al constructor de la clase padre mediante super.

También se debe agregar la operación de creación en Copa y ControladorEncuentros, y una opción en la vista para seleccionarlo. La tabla de posiciones y el calendario no necesitan modificarse: la tabla utiliza los puntos acumulados de los equipos y el calendario trabaja con objetos que cumplen ICalendario. El nuevo deporte participa en ambos mediante el comportamiento heredado y el polimorfismo.

Diseño

